Measuring Verifier Impact in an LLM Generate→Verify→Repair Loop for Intent→Configuration NetOps Tasks

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract*—We measure how inserting a deterministic network verifier into a generate→verify→repair (G→V→R) loop affects verifier-detected safety violations for LLM-produced device-configuration proposals. In a preregistered paired design (N = 500 intent→configuration tasks stratified across VLAN-change, ACL-change, and static-route-change clusters) we compared: (1) baseline — a single LLM proposal produced once per task, and (2) guarded — the identical shared LLM proposal validated by a deterministic netops verifier and subjected to up to one automated repair iteration if the verifier flagged a preregistered high/medium-severity violation. Primary outcome: verifier pass (no preregistered high/medium-severity violations). Results (complete pairs = 500): baseline passed 499/500 (0.998), guarded passed 500/500 (1.000); contingency: n11 = 499, n10 = 1, n01 = 0, n00 = 0. Absolute paired success-rate difference (guarded - baseline) = 0.002; paired-bootstrap 95% percentile CI = [0.0, 0.006] (B = 10,000, seed = 1729). The preregistered exact McNemar two-sided test was implemented as an exact binomial on discordant pairs (n = n10 + n01); with n = 1 and n10 = 1 the archived two-sided p = 1.00 under that convention. Interpretation: on this verifier-scoped synthetic corpus and model+validator pairing the LLM produced near-ceiling proposals; measured marginal verifier+repair effect on the pass-rate metric is therefore small, though the verifier corrected at least one concrete baseline failure. All preregistration, prompts, model outputs, validator traces, provider-call traces, execution logs, and analysis artifacts are archived in the evidence bundle for deterministic re-inspection.

I. Introduction

Generative large language models (LLMs) are being applied to NetOps tasks such as translation of operator intent to device configuration, multi-step change planning, and programmatic repairs. Operational safety in these systems depends heavily on surrounding engineering: deterministic validators, sandbox replay, canarying, and rollback mechanisms are commonly recommended and instantiated in practice. A frequently proposed operational pattern is generate→verify→repair (G→V→R): an LLM generates a candidate configuration, a deterministic verifier checks a set of formalized properties, and automated or human-in-the-loop repairs are applied for violations.

Despite conceptual consensus about the value of guardrails, controlled measurements that isolate the marginal contribution of a verifier (holding the model's proposal fixed) are scarce. Practitioners therefore lack quantitative guidance on whether, and by how much, a verifier plus a modest automated repair budget improves verifier-scoped safety when the LLM already produces high-quality candidates. We preregistered and executed a paired experiment that isolates verifier+repair impact under canonical deterministic semantics. Our objective was deliberately narrow and operational: given identical initial proposals (shared-initial_candidate semantics), how much does integrating a deterministic verifier and at most one automated repair iteration reduce verifier-detected safety violations compared to the same LLM alone?

II. Related Work

Network-configuration verification has a long history in systems and formal-methods communities. Deterministic analyzers (e.g., header-space and route-model verifiers, Batfish-style tools) can check reachability, isolation, and update-safety properties and have been shown to catch subtle misconfigurations pre-deployment. Concurrently, recent NetOps and AIOps work demonstrates LLMs for intent parsing and configuration generation while arguing that operational guardrails are essential for reliability. Generate–verify–repair or generate–critic patterns appear across structured-generation domains and are proposed to reduce unsafe outputs. Separate literature documents systemic failure modes of LLM-driven automation (hallucination, overconfidence, tool-integration errors) and recommends verification or human oversight as mitigations. What is less prevalent in prior published work is a controlled, preregistered ablation that quantifies the marginal verifier effect while keeping the LLM's initial output identical for both arms; our study contributes such an artifact-backed measurement and situates it relative to the verification and agentic-NetOps literature.

III. Methodology

Preregistration and inferential plan. We preregistered the study (study_id cand_7f4b9a2d_hosted_gvr_v1_2026) and froze inclusion rules, random seeds, estimands, analysis procedures, multiplicity adjustments, and a stopping rule before observing outcomes. The primary confirmatory hypothesis compared paired binary outcomes (success == verifier reports zero preregistered high/medium-severity violations) between baseline and guarded conditions. The prespecified primary estimand was the paired_success_rate_difference_guarded_minus_baseline.

Task corpus and stratification. A deterministic synthetic corpus of 500 intent→configuration tasks was generated with fixed seeds and stratified into three clusters: VLAN-change

(167), ACL-change (167), and static-route-change (166). Generator constraints limited modeled fields to {admin, mtu, vlan} to match verifier coverage. No preregistered confirmatory exclusions occurred.

Generation semantics and experimental control. To isolate verifier marginal effect we used shared_initial_candidate semantics: for each task the pipeline produced a single shared LLM proposal that served as both the baseline output and the input to the guarded arm. The guarded arm invoked an automated single-repair attempt only when the deterministic verifier flagged a preregistered high- or medium-severity violation on that shared candidate. This design prevents conflation with verifier-guided regeneration and isolates the effect of verification plus at most one repair applied to the same initial proposal.

Modeling and verifier. The requested LLM was gpt-5-mini called through a hosted adapter with single-attempt generation and a 2000-token output limit. The deterministic verifier used for gating and scoring was a purpose-built netops validator that maps concrete violation codes to severity levels. The guarded arm permitted at most one automated repair iteration per preregistered contract.

Confirmatory test and resampling. The preregistered confirmatory test was an exact McNemar two-sided test on paired binary outcomes implemented as an exact binomial on the discordant-pair count (n = n10 + n01) with the two-sided p-value computed by doubling the smaller tail probability and truncating at 1.0. Paired-bootstrap percentile 95% confidence intervals for the absolute paired difference used B = 10,000 resamples with seed = 1729 and preserved pairing during resampling. Resampling unit = task/pair.

Operational controls and accounting. The preregistered retry policy allowed limited automated retries for transient provider errors; execution stopped after processing the preregistered confirmatory pairs. Execution accounting recorded the number of completed episodes, model calls, and observed repair-stage calls. All design choices, seeds, and analysis code were preregistered prior to execution.

IV. Results

Execution summary and primary outcomes. The run processed 500 complete pairs under shared_initial_candidate semantics. Observed model-call accounting shows 501 hosted model calls: 500 shared-initial generations and 1 repair-stage call. Aggregated per-condition results: baseline successes = 499, baseline failures = 1 (baseline_success_rate = 0.998); guarded successes = 500, guarded failures = 0 (guarded_success_rate = 1.0). The paired contingency counts were n11 = 499 (both pass), n10 = 1 (guarded pass only), n01 = 0 (baseline pass only), n00 = 0 (both fail). Absolute paired success-rate difference (guarded - baseline) = 0.002.

Inferential results. Under the preregistered exact-binomial-on-discordant-pairs convention the two-sided McNemar p-value for the observed discordant counts (n = 1, n10 = 1) is p = 1.00. The paired-bootstrap percentile 95% CI for the absolute difference is [0.0, 0.006] (B = 10,000, seed = 1729). These archived analyses preserve the preregistered test definitions and implementation choices.

Repair activity and discordant episode provenance. The guarded arm invoked a single repair-stage call; that repair produced the sole discordant outcome (the one task where the baseline failed and the guarded arm passed). The run-level accounting (model_calls_used = 501; observed repair-stage calls = 1) and the per-task scoring traces confirm exactly one such corrected episode in the confirmatory set. All per-task validator diagnostics, repair prompts/responses, and provider-call traces are archived in the evidence bundle for deterministic inspection; the archived artifacts record the validator violation codes and the repair content for that episode.

Secondary analyses and power considerations. The preregistration power calculation assumed an anticipated baseline failure prevalence $\approx 30\%$ to achieve 80% power for detecting a 10-percentage-point absolute reduction ($N \approx 384$). Observed baseline failure prevalence (1/500) was far lower than anticipated, producing a ceiling effect that substantially reduced sensitivity to detect small absolute improvements. Consequently, secondary per-violation-class analyses were uninformative due to near-zero counts.

Representative task examples. Representative task records from the run illustrate the task patterns and validator outcomes. Many tasks (including medium- and hard-pattern examples) produced identical baseline and guarded proposals that the verifier accepted; the single corrected episode demonstrates that a modest repair budget can convert a rare baseline failure into a verified pass. The evidence bundle contains the full per-task validator diagnostics and normalized configurations supporting these statements.

V. Discussion

Conditional interpretation. Under the constrained experimental regime — single requested LLM (gpt-5-mini requested), shared_initial_candidate semantics, a deterministic verifier modeling a limited feature set (allowed_fields = {admin, mtu, vlan}), and at most one automated repair iteration — the LLM produced near-ceiling correct proposals on the verifier-scoped properties and the measured marginal pass-rate benefit of adding the verifier was small (0.002). This is a conditional, operationally framed measurement: when baseline proposal quality is very high relative to the verifier's coverage, modest verifier+repair budgets yield limited measurable gains on a composite pass/fail metric, yet can still remediate rare concrete failures, as observed here.

Design and deployment implications. The shared_initial_candidate semantics intentionally answer the operational question of marginal verifier benefit when regeneration costs are constrained or when deployment semantics require preserving the original proposal for human review and repairing it instead of regenerating. Alternative operational choices — verifier-guided regeneration, expanded repair budgets, or different cost trade-offs — may yield different effect sizes and should be measured under separate preregistrations adapted to those semantics.

Threats to validity and generalization. Internal validity is supported by preregistration, deterministic task seeds, and automated execution controls. Construct validity is limited by the verifier's modeled property set: success (no high/medium-severity violations) is defined relative to the verifier's coverage and severity mapping. If operators care about properties outside that modeled set (vendor-specific CLI idiosyncrasies, protocol state beyond the verifier's model), the measured pass-rate difference underestimates the operational benefit of broader verifier coverage. External validity is limited by single-model testing, a synthetic deterministic task corpus, and constrained feature scope; we do not claim cross-model, cross-vendor, or unconstrained-production generality.

Statistical nuance. The archived exact McNemar two-sided p = 1.00 reflects the preregistered exact-binomial convention and the small number of discordant pairs (one). Small-sample exact p-values can be non-intuitive; here p = 1.00 indicates the observed pattern is not statistically unusual under the null when discordant counts are so small. The paired-bootstrap CI [0.0, 0.006] provides complementary uncertainty quantification for the absolute paired difference. Together these results show a small point estimate improvement (0.002) but wide relative uncertainty and low power due to the ceiling effect.

Practical recommendations. First, measure baseline proposal error prevalence for verifier-scoped properties on representative operational data before allocating repair budgets: extremely low baseline error rates limit marginal return on modest verifier+repair investments. Second, align verifier coverage with operator-critical constraints and expand the verifier's modeled properties if necessary. Third, choose generation semantics to match deployment priorities: shared-initial semantics minimize model-call overhead and preserve the original proposal for human review, whereas verifier-guided regeneration or larger repair budgets may increase final-pass probability at additional model-call cost.

VI. Limitations

- Synthetic deterministic task corpus limited to the verifier-modeled features; task payloads were constrained to allowed_fields = {admin, mtu, vlan}. Results do not generalize to unconstrained production intents, wider cross-vendor CLI heterogeneity, or dynamic protocol states outside the verifier scope.
- Single requested model family (gpt-5-mini requested) and a single deterministic validator (deterministic_netops_validator_v5) — we do not claim multi-model or cross-validator generality.
- Maximum one automated repair iteration was permitted by the preregistration; different repair budgets or iterative regeneration strategies could change measured verifier impact.
- Very high baseline pass rate (499/500) induced a ceiling effect that limits power to detect small absolute improvements on the pass-rate metric; the archived exact McNemar two-sided p = 1.00 reflects the preregistered exact-binomial-on-discordant-pairs convention and the small discordant count.
- The single discordant episode (the sole task where baseline failed and guarded succeeded) and its validator diagnostics and repair-stage trace are archived in the study evidence bundle. The run-level manifest and per-task scoring traces confirm exactly one such corrected episode in the confirmatory set.

VII. Conclusion

We conducted a preregistered, artifact-backed paired experiment (N = 500) that isolated the verifier+single-repair effect under shared_initial_candidate semantics. The LLM produced near-ceiling verifier-scoped proposals (baseline 499/500; guarded 500/500), giving an absolute improvement of 0.002 (95% CI [0.0, 0.006]; archived exact McNemar two-sided p = 1.00 under the run's exact-binomial convention). This conditional measurement illustrates a ceiling-effect regime: when baseline proposal quality is already very high for the verifier-modeled properties and verifier coverage is limited, the measurable marginal pass-rate benefit of a modest verifier+repair budget can be negligible on a composite pass/fail metric, while still correcting rare concrete failures. Practitioners should estimate baseline error prevalence on representative workloads, align verifier property coverage with operational priorities, and select generation/repair semantics that match deployment constraints. All preregistration, prompts, model outputs, validator traces, provider-call logs, execution logs, and analysis artifacts are archived in the study's evidence bundle for deterministic re-inspection of the claims above.

Disclosure Statement

This study was generated and executed autonomously under an archived initial master prompt for the run; the immutable archived master-prompt file is recorded as provenance/master_prompt.txt (SHA-256 = 1872df1e 1805d2d96940456ca016bd665d1d5196add77f5acdf1582b b39b15ba). Human involvement: a human Research Director selected the broad topic family and constrained the initial master prompt before the autonomy lock. After the autonomy lock, all literature discovery, research-question selection, preregistration, experiment design, execution, analysis, manuscript drafting, internal agentic peer review, and revisions were performed autonomously by the agentic research pipeline. Requested model: gpt-5-mini (provider recorded as openai_responses); deterministic verifier: deterministic_netops_validator_v5. The preregistration preceded execution and was archived prior to data collection. Execution accounting recorded 500 complete task pairs, 501 hosted model calls (500 shared-initial generations + 1 repair-stage call), and exactly one observed repair application. All experimental artifacts required to rerun or audit the study (preregistration, task generator, model responses, validator traces, provider-call logs, and analysis outputs) are archived in the study evidence bundle.

REFERENCES

[1] https://doi.org/10.1145/3098822.3098834
[2] https://doi.org/10.1145/3718958.3750537
[3] http://arxiv.org/abs/2309.05557
[4] https://doi.org/10.2139/ssrn.6484501
[5] https://arxiv.org/abs/2605.12729
[6] Muhammad Bilal, Jon Crowcroft, Ruizhi Wang, Xiaolong Xu, Schahram Dustdar, "Large Language Models for Agentic NetOps and AIOps: Architectures, Evaluation, and Safety", 2026, 10.48550/arxiv.2605.12729
[7] Yunze Wei, Xiaohui Xie, Tianshuo Hu, Yiwei Zuo, Xinyi Chen, K.N. Chi, Yong Cui, "INTA: Intent-Based Translation for Network Configuration with LLM Agents", 2025, 10.1109/icnp65844.2025.11192391